CSc 453 — Assignment 7

Christian Collberg
University of Arizona, Department of Computer Science

April 9, 2026

1 Introduction

Your job is to extend the interpreter from the previous assignment to handle procedure calls.

2 Theory Questions

Each question is worth 4 points.

Some of the problems require you to submit your answer in the form of a Graphviz file. You can learn about Graphviz by watching lecture **misc-1** in `https://ligerlabs.org/lectures.html`.

2.1 Problem 1

Consider the famous Ackermann's function:

```
int ackermann(int m, int n) {
/* [0] */    if (m == 0) {
/* [1] */        return n + 1;
/* [2] */    } else if (n == 0) {
/* [3] */        return ackermann(m - 1, 1);
/* [4] */    } else {
/* [5] */        int a = ackermann(m, n - 1);
/* [6] */        int b = ackermann(m - 1, a);
/* [7] */        return b;
/* [8] */    }
/* [9] */
}
int main() {
/* [10] */  int a = 2;
/* [11] */  int b = 0;
/* [12] */  int r =  ackermann(a,b);
/* [13] */  printf("%i\n", r);
/* [14] */  return 0;
}
```

1. Draw the call tree for `ackermann(2,1)`. Enter your answer in the file `problem1.gv`.
2. Show how the call stack changes during the execution of `ackermann(2,0)`. For each activation record show: actual parameters, local variables, return address, and return value. For the return address, use the line numbers shown in brackets. Enter your answer in the file `problem1.json`.

2.2 Problem 2

For the C program below show the *environment* at points A, B, C, D, E. Point E has been given to you. Enter your answer in the file `problem2.json`.

```
int i;

void P(int a) {
   double x;
   void Q(float b) {
      short c;
      void R(long d) {
         /* POINT_A */
      }
      void S(long e) {
         int f;
         {
            float g;
            /* POINT_B */
         }
      }
      /* POINT_C */
   }
   void T(long h) {
      unsigned long i;
      /* POINT_D */
   }
}

int main() {
   float j;
   /* POINT_E */
}
```

3 Getting Started

This assignment can be done in teams of one or two.

To get started, follow these instructions:

1. Unzip `assignment-7.zip` which gives you these files:

   ```
      assignment-7/assignment-7.pdf
      assignment-7/firstname_lastname/tests/*.luc
      assignment-7/firstname_lastname/vm_files/*.out
      assignment-7/firstname_lastname/myprog.luc
      assignment-7/lucac_macos_m1
      assignment-7/lucac_linux_x86_64
      assignment-7/firstname_lastname/collaboration.json
      assignment-7/firstname_lastname/survey.json
    survey
   ```

2. Then, assuming your name is Pat Jones, execute these commands:

```
> cd assignment-7
> mv firstname_lastname pat_jones
```

If you are working in a group of two, pick one of your names.

3. **Fix bugs from the previous assignment!** If your interpreter can't handle arithmetic, control flow, etc., you may lose points when we are grading your ability to handle function calls.
4. Extend your switch-based interpreter from the previous assignment (`luca_run_switch.c`) to handle procedure declarations and procedure calls.
5. Write your own Luca program in `firstname_lastname/myprog.luc`.

4 Implementation

Your task is to write an interpreter for the language Luca.

1. The interpreter should be named `luca_run_switch`.
2. You first compile your program to LVM code using the provided Luca front-ends:

```
> lucac_macos_m1     -i x.luc -v x.vm
> lucac_linux_x86_64 -i x.luc -v x.vm
```

And then you run your interpreters on that vm code:

```
> luca_run_switch x.vm
```

3. **This assignment should be coded in C**.

5 LVM Bytecode File

Consider this program:

```
PROGRAM P;

PROCEDURE Q (
   a : INTEGER;
   b : CHAR;
   c : REAL);
VAR d : INTEGER;
BEGIN
   WRITE a+d;
END;

VAR x : INTEGER;
VAR y : REAL;

BEGIN
   Q(x, '0', y);
END.
```

This is the corresponding LVM file:

```
55 16          # [0] 0: DEF_GLOBALS

# Procedure Q starts here
38 24 8          # [1]  0: PROC_BEGIN
4 0              # [2] 17: PUSHADDR_formal
19               # [3] 26: LOAD_i
3 0              # [4] 27: PUSHADDR_local
19               # [5] 36: LOAD_i
8                # [6] 37: PLUS_i
44               # [7] 38: WRITE_i
39 24 8          # [8] 39: PROC_END

# The main program starts here
1                # [9]  56: PROG_BEGIN
5 0              # [10] 57: PUSHADDR_global
19               # [11] 66: LOAD_i
42               # [12] 67: ACTUAL_i
18 48            # [13] 68: PUSHCONST_i
42               # [14] 77: ACTUAL_i
5 8              # [15] 78: PUSHADDR_global
33               # [16] 87: LOAD_f
43               # [17] 88: ACTUAL_f
40 -89 24        # [18] 89: CALL
2                # [19] 106: PROG_END
```

Anything following the `#`-character is a comment. You should not use this information in your interpreter; it's just there to help us understand the bytecode.

Procedures are referenced by their address in the bytecode. Procedure `Q`, for example, is at address 0. The call to `Q` are the bytes `40 -89 24`, where 40 is the byecode for `CALL`, `-89` is the address (relative to the current bytecode instruction, which is 89) of `Q`.

6 The Luca Virtual Machine (LVM)

Opcodes that end in `_i` work on integers, those that end in `_f` on reals.

For simplicity, all basic types (`INTEGER`, `REAL`, `CHAR`, `BOOLEAN`) are 8 bytes wide. I.e., `INTEGER` and `REAL` correspond to `long` and `double`, respectively, in C on lectura.

Note that

- procedures can be recursive;
- procedures cannot be nested.

Instruction	Opcode	Arguments	Stack Pre	Stack Post	Description
PROC_BEGIN	38	LocalSz FormalSz			Procedure header.
PROC_END	39	LocalSz FormalSz			Procedure footer.
CALL	40	Addr FormalSz	[]	[ ]	Call the procedure at Addr. Arguments have been previously provided through ACTUAL_f and ACTUAL_d instruvtions. Addr is the offset of the first instruction (the PROC_BEGIN instruction) of the called procedure, with respect to the address of this (CALL) instruction.
ACTUAL_i	42		$[A]$	[]	Pass A as an argument to the next call.
ACTUAL_f	43		$[A]$	[]	Pass A as an argument to the next call.

7 Academic Integrity

- Don't show your code to anyone outside your team.
- Don't read anyone else's code.
- Don't discuss the details of your code with anyone.
- If you need help with the assignment see the TAs or the instructor.

7.1 Use of AI

You must write all code by hand without the use of any AI tools.

Q: Can I upload the assignment to an AI tool to explain it to me?

A: No. If you don't understand the assignment, ask the TAs.

Q: Can I use an AI tool to generate test cases?

A: No.

Q: In fact, are there *any* possible use cases for which it is OK for me to use AI tools in this assignment?

A: No, there are *no* such use cases.

Q: I will be using AI coding tools in my future job. So why can't I prepare for that by using such tools in this class?

A: This is a compiler class. The goal is to learn about compilers. If you want to learn about AI, take an AI class.

Q: But you will never be able to tell if my code was generated by an AI coding tool!

A: ROTFL!

8 Submission

To submit, follow these instructions:

1. Verify that your submission contains a working `Makefile` and *all* the files necessary to build the interpreters. Your program must compile out of the box. The graders will *not* try to debug your program or your makefile for you.
2. You should test the interpreter on lectura.
3. Fill out `collaboration.json` with information about your group.
4. Optionally fill out `survey.json` to give me feedback about the assignment.
5. Package up your answers and verify that they look OK:

```
> zip -r pat_jones.zip pat_jones
> assignment-7/validator.sh pat_jones.zip
==== This is the LigerLabs assignment validator ====
          ...        ...        ...        ...
==== 6: Checking that all problems have been answered.
==== 7: Checking that all answer files have valid structure.
```

6. Upload `pat_jones.zip` to d2l. If you are working in a group of two, you should only submit once.

9 Grading

21 test cases have been provided for you in `assignment-7/tests/*/*.luc`. The corresponding vm files are in `vm_files/*/*.vm`.

When we run the test cases we will remove everything after the `#`-character to make sure that you don't make use of the information in the comments in your interpreter.

Each test case will give you 0 points if you get it wrong, 3 point if you get it right.

The theory part is worth 8 points.

You can get an additional 14 points from secret test cases.

You can get an additional 15 points for a *unique* and *interesting* Luca program that you have written yourself. It should be at least 50 lines long (not counting blank lines and comments) and it should have at least 3 procedures. Feel free to reuse a program from another class that you've translated to Luca.